

Niranjan N Prabhu

draft.docx

-  Plagiarism Check
-  NIRANJAN PHD
-  Manipal Academy of Higher Education (MAHE)

Document Details

Submission ID

trn:oid::1:3639406545

Submission Date

Sep 3, 2026, 4:56 PM GMT+8

Download Date

Sep 3, 2026, 5:08 PM GMT+8

File Name

draft.docx

File Size

1.8 MB

30 Pages

7,554 Words

47,163 Characters

51% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely revised by AI.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.



65 AI-generated 51%

Likely generated or likely generated and revised by AI.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (it may misidentify writing that is likely human generated as AI generated and likely AI generated as human generated) so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Biometric-Based Key Sequence Generation Using Genetic Algorithms for Enhanced Blockchain Security

Fragan D'souza

Abstract

This paper presents the design, implementation, and experimental evaluation of a biometric cryptographic key generation and blockchain integrity verification system, developed as an implementation and extension of the facial landmark based key generation approach proposed by Sannidhan et al. [1]. The base paper combines a MobileNetV2 based facial landmark regressor, a genetic algorithm (GA) for LFSR parameter optimization, and an 8-register linear feedback shift register (LFSR) to generate cryptographic key material, reporting a dataset level Shannon entropy of 3.24 bits over 92,706 generated sequences. Building on this foundation, the present work implements a MediaPipe Face Mesh based landmark extraction pipeline (with a deterministic synthetic fallback), Procrustes feature normalization, a multi-objective GA for feature selection, an independently configured 32-bit Galois LFSR key material expansion stage, RFC 5869 HKDF-SHA256 key derivation, AES-256-GCM authenticated encryption, and a proof of work blockchain layer that records only ciphertext hashes and never raw biometrics or secret keys. Neither the encryption nor the blockchain component exists in the base paper; both are original additions of this project. The implemented system was evaluated from start to finish: the full pipeline (106 landmarks → 212 normalized features → GA feature selection → LFSR/HKDF key derivation → AES-256-GCM encryption → blockchain integrity verification) executed successfully with a verified decrypt and compare round trip. A five-way baseline comparison measured byte level Shannon entropy (approximately 7.94 bits/byte, near the 8-bit theoretical maximum) and Wald-Wolfowitz runs test statistics across SHA-256, HKDF, GA-HKDF, and the proposed GA-LFSR-HKDF architectures, alongside a faithful re-implementation of the base paper's 8-register LFSR mechanism for direct comparison. A biometric stability study across ten synthetic subjects (five captures each) measured a 24% false rejection rate and a 0% observed false acceptance rate at the evaluated decision threshold, indicating a genuine reproducibility challenge for regenerating keys from biometrics that is disclosed rather than minimized. The results demonstrate a functioning integration of biometric feature selection, cryptographic key derivation, authenticated encryption, and blockchain based integrity auditing, while identifying concrete limitations, including the FRR/FAR tradeoff, the deterministic nature of LFSR based expansion, and the absence of a fuzzy extractor mechanism, as targets for future work.

Keywords: Biometric Cryptography; Genetic Algorithm; Facial Landmark Extraction; Linear Feedback Shift Register; HKDF-SHA256; AES-256-GCM; Blockchain Integrity Verification; Key Derivation; Biometric Key Stability

1. Introduction

1.1 Background

Cryptographic systems ultimately depend on the secrecy and unpredictability of their key material. Traditional key management, whether passwords, hardware tokens, or externally stored random seeds, introduces usability and custody burdens: keys must be remembered, stored, or physically carried, and each of these carries its own risk of loss, theft, or simply being forgotten. Biometric key generation attempts to bind cryptographic material to a person's physiological characteristics so that key material can, in principle, be regenerated from the person rather than retrieved from storage. Sannidhan et al. [1] proposed one such approach for facial biometrics, combining deep learning based landmark extraction, genetic algorithm parameter optimization, and an LFSR based sequence generator to produce key material with favorable statistical randomness properties.

1.2 Problem Statement

The base paper [1] addresses key-sequence generation from facial features in isolation, without specifying how the generated key material would be used for actual data protection or how the integrity of protected data would be independently verifiable over time. This project addresses the downstream problem: given biometrically derived key material, how should it be used to encrypt real data with authenticated encryption, and how can the integrity of that encrypted data be verified by a third party without exposing the plaintext, the key, or the underlying biometric.

1.3 Motivation

A dynamic, user-specific key derived from biometric features avoids the need to store a long-term secret key directly. Combining such a key with authenticated encryption (to protect confidentiality and detect tampering at decryption time) and a tamper evident ledger (to detect tampering independently of decryption, via publicly verifiable hash commitments) extends the base paper's scope from "key generation" to "key generation plus a usable, auditable data protection workflow."

1.4 Research Gap

Three gaps motivate this work. First, the base paper does not specify or evaluate any downstream cryptographic use of the generated key material: no encryption scheme, key derivation step, or authenticated encryption evaluation is presented. Second, it does not address data integrity auditing: there is no mechanism by which a third party could later verify that protected data had not been altered. Third, the base paper's own entropy metric (3.24 bits, computed over a symbol alphabet specific to its LFSR register representation) is not evaluated using standard byte level randomness tests (Shannon entropy in bits/byte, Wald-Wolfowitz runs test) that would allow direct comparison against conventional cryptographic key derivation baselines such as HMAC-based Extract-and-Expand Key Derivation (HKDF).

1.5 Objectives

- Objective 1: Develop a secure cryptographic framework that generates dynamic, user-specific encryption key material from facial biometrics using Genetic Algorithms, with emphasis on uniqueness, robustness, stability, and entropy.

- Objective 2: Use the generated cryptographic key material for data encryption and integrate the encrypted-data workflow with blockchain operations for immutable/decentralized data management and integrity verification.
- Objective 3: Address limitations of conventional cryptographic approaches by combining biometrically driven key generation, adaptive optimization, cryptographic key derivation, authenticated encryption, and blockchain based integrity verification.

1.6 Contributions

- An end-to-end, code-verified pipeline integrating facial-landmark feature selection (via a multi objective GA), LFSR based key material expansion, HKDF-SHA256 key derivation, AES-256-GCM authenticated encryption, and a proof-of-work blockchain integrity ledger.
- A five-way controlled comparison isolating the marginal statistical effect of each pipeline stage (SHA-256 only; HKDF only; GA + HKDF; the full proposed GA-LFSR-HKDF chain; and a faithful re-implementation of the base paper's own 8-register LFSR mechanism) under an identical measurement methodology.
- An explicit, code-verified account of every implementation adaptation from the base paper, including the landmark extraction substitution and the two functionally distinct LFSR mechanisms used in this codebase.
- An honest biometric-stability evaluation (intra-/inter-subject distance, FRR, FAR) that discloses the current reproducibility limitation rather than presenting the system as production-ready authentication.

Within the scope of this project, these contributions are best characterized as an engineering integration and extension of an existing key generation technique into a complete, auditable data protection workflow, rather than as a new cryptographic primitive.

2. Related Work

This section summarizes the prior work identified and cited by the base paper [1], which forms the scientific lineage of the key generation technique this project builds on.

2.1 Biometric Cryptography

Early work explored combining cryptography with biometrics directly, using fingerprint sensors, passwords, or facial images as sources of key material [2], [3]. A recurring limitation across this line of work is dependence on additional hardware (e.g., fingerprint sensors) or on features that are not robust to environmental capture noise.

2.2 Facial Feature-Based Key Generation

Deep learning based facial feature extraction has been used both for privacy protection in cloud platforms [2] and, more directly, as a source of key material for stream ciphers using SURF feature descriptors [4], the authors' own earlier system, which the base paper explicitly improves upon by replacing SURF based extraction with a trained MobileNetV2 landmark regressor to reduce repetitive sequence generation.

2.3 Genetic Algorithms in Feature Optimization

Genetic algorithms have been applied to pseudo-random number generation [5], DNA-cryptography key generation [6], and elliptic-curve key optimization [7]. The base paper's own comparison baseline, Sudeepa et al.'s GA based key sequence generator [8], is used directly as one of the base paper's quantitative comparison points and is likewise used conceptually here to frame the GA-HKDF baseline evaluated in Section 7.

2.4 LFSR-Based Pseudo-Random Generation

Linear feedback shift registers remain a common building block for lightweight sequence generation, including LFSR with Gaussian distributed feedback [9] and LFSR driven 3D Playfair ciphers [10]. A widely known property of LFSRs, acknowledged throughout this literature, is that their output is fully determined by the register's initial state and feedback structure. An LFSR alone provides no cryptographic secrecy if an attacker recovers the seed and taps.

2.5 Cryptographic Key Derivation

Standardized key derivation functions such as HKDF (RFC 5869) [12] are designed to convert non uniform input key material into cryptographically strong, uniformly distributed key bytes via an HMAC based Extract-and-Expand construction. HKDF is not evaluated or used anywhere in the base paper [1]; its use in this project is an original addition intended to standardize the final key derivation step regardless of upstream biometric noise.

2.6 Authenticated Encryption

AES-256 in Galois/Counter Mode (GCM), standardized in FIPS 197 [13] and NIST SP 800-38D [14], provides both confidentiality and integrity/authenticity in a single primitive via an authentication tag. Like HKDF, AES-256-GCM does not appear anywhere in the base paper and is an original addition of this project.

2.7 Blockchain-Based Integrity

Blockchain ledgers provide tamper evident, append-only records through cryptographic hash chaining and proof of work consensus. Using a blockchain purely as a metadata or integrity layer, storing only a hash commitment to encrypted data rather than the data itself, is a common pattern for combining off-chain confidentiality with on-chain auditability. This layer, too, is entirely absent from the base paper and is an original addition of this project.

2.8 Research Gap

No work identified in the base paper's own literature review, nor the base paper itself, evaluates the combination of (i) GA-optimized biometric feature selection, (ii) standardized HKDF key derivation, (iii) authenticated encryption, and (iv) blockchain based integrity verification as a single, end-to-end, experimentally validated pipeline. This is the gap this project addresses.

3. Base Paper and Motivation

3.1 Base Paper Methodology

Sannidhan et al. [1] (“Genetic algorithms and deep learning for unique facial landmark-based key generation,” *Computers and Electrical Engineering*, vol. 118, article 109427, 2024) propose a three-module pipeline. A MobileNetV2 convolutional neural network, trained on 224×224 input images with the Adam optimizer (learning rate 0.0001), regresses 106 facial landmark coordinates directly from a face image. The (x, y) coordinates of each landmark are fused into a single scalar via the Cantor pairing function. A genetic algorithm (population size 100, 1000 generations, two-point crossover at rate 0.8, random-resetting mutation at rate 0.05, steady-state parent selection) then optimizes 8 register seed values for an LFSR. The LFSR itself holds 8 registers $X_0 \dots X_7$; each iteration computes a new value from the sum of the even-indexed registers modulo 256, shifts all registers down by one position, and inserts the new value into X_7 , using feedback polynomial $x^7 + x^5 + x^3 + x + 1$. Iterating this process for each of 74 distinct GA-optimized seed pairs produced 92,706 total sequences (up to 1287 sequences per seed) across four public face datasets (LFW [16], FERET [17], Extended Yale Face B [18], CMU Multi-PIE [19]).

3.2 Relevant Characteristics

The base paper reports a Shannon entropy of 3.24 bits for its generated sequence dataset (computed over the register value alphabet used by its own LFSR representation, not a byte level entropy in the 0 to 8 bits/byte sense used elsewhere in cryptographic randomness testing), an average Pearson correlation coefficient of 0.1554 across ten sample sets, and a runs test Z-statistic of 1.6083. It compares its total sequence and Z-statistic figures against three prior LFSR based systems, including its own earlier SURF feature based system [4].

3.3 Limitations / Opportunities Identified

The base paper is explicitly scoped to secret-key generation; it states in its own conclusion that no storage, transmission, or downstream cryptographic use security is addressed, and lists quantum-safe key distribution, cross-modal biometric extension, and industry deployment as future work. It also discloses that its datasets and code are not publicly available (“The authors do not have permission to share data”), which motivates an independent re-implementation rather than direct reuse of the original artifacts.

3.4 Motivation for Our Adaptation

Given the above, this project treats the base paper's GA plus LFSR key generation concept as a starting point, re-implements its core mechanism for direct experimental comparison (Section 5.5), and then extends the pipeline downstream into an actual data protection workflow: HKDF key derivation, AES-256-GCM encryption, and blockchain based integrity verification, none of which exist in the cited work.

4. Proposed System

4.1 System Overview

The implemented system takes either a face image or a synthetic/direct landmark array as input and produces an authenticated ciphertext together with a blockchain transaction recording a tamper evident hash commitment to that ciphertext. Figure 1 shows the ten pipeline stages in order.

Figure 1. Overall System Architecture

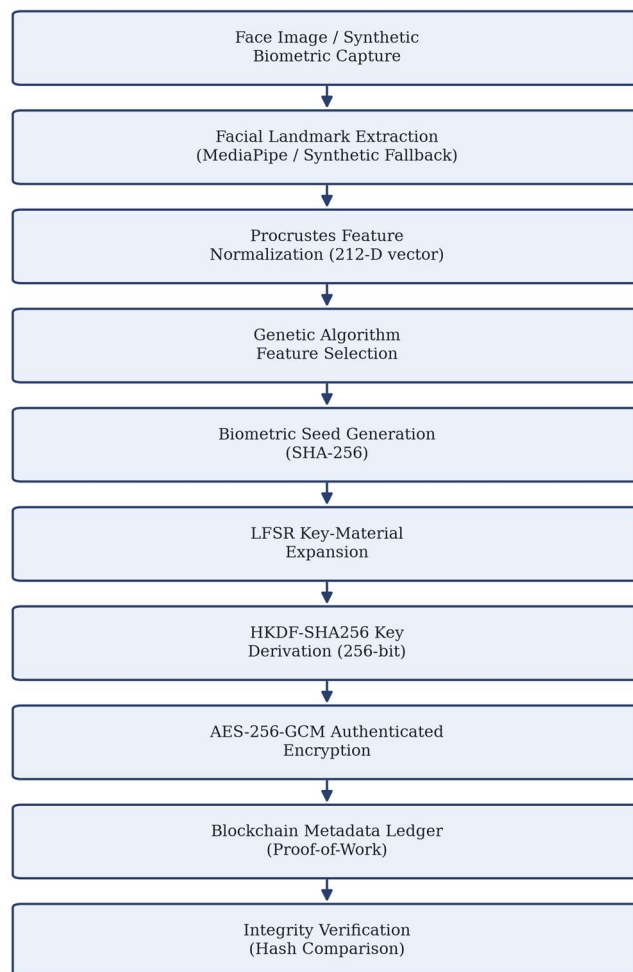


Figure 1. Overall system architecture.

4.2 Overall Architecture

The pipeline is organized into five functional groups: biometric feature extraction and normalization (Sections 4.3 and 4.4), genetic algorithm feature optimization (Section 4.5), key material generation (Sections 4.6 through 4.8), authenticated encryption (Section 4.9), and blockchain integrity management (Sections 4.10 and 4.11).

4.3 Facial Feature Extraction

Facial landmarks are extracted using Google MediaPipe Face Mesh, which regresses 468 3D landmark points from an input image. This codebase uniformly downsamples those 468 points to 106 canonical coordinates via NumPy's linspace based index selection. If MediaPipe is unavailable or fails to detect a face, a deterministic synthetic 106-point anatomical landmark generator is used as a fallback, built from image-derived statistics as a seed. Figure 2 details this stage together with the normalization stage that follows it.

Figure 2. Biometric Processing Pipeline

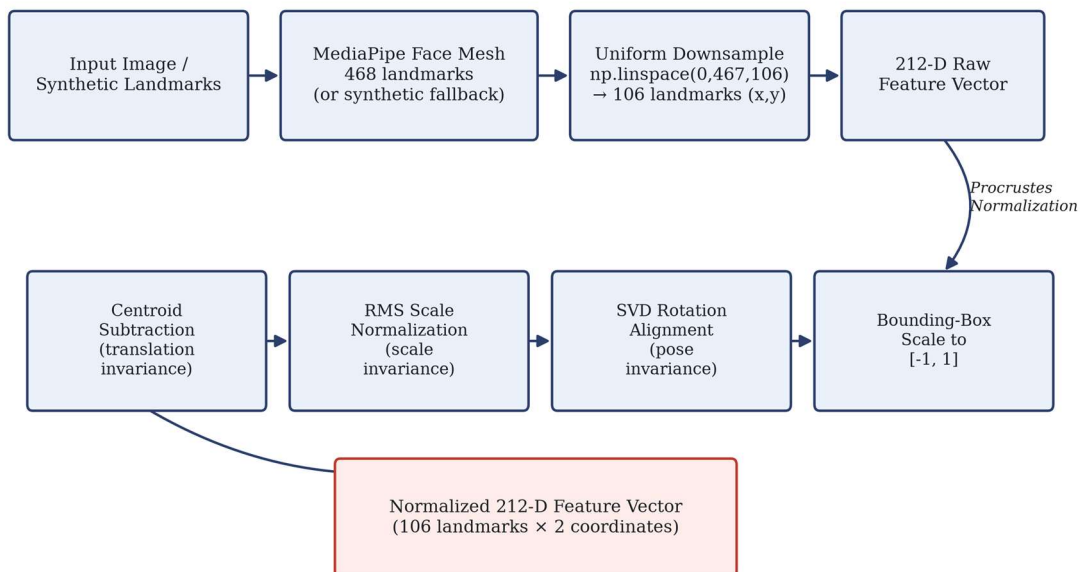


Figure 2. Detailed biometric processing pipeline.

4.4 Feature Normalization

The 106 (x, y) landmark coordinates are normalized using a Procrustes-style transform: centroid subtraction removes translation, root-mean-square (RMS) distance scaling removes overall scale, and singular value decomposition (SVD) of the landmark covariance matrix estimates and removes a global in-plane rotation. Coordinates are then rescaled to the fixed bounding box range $[-1, 1]$. The resulting $106 \times 2 = 212$ -dimensional vector is the normalized feature representation used by every downstream stage.

$$\mu = \text{mean}(P); \quad P' = P - \mu \quad (\text{centroid subtraction})$$

$$s = \sqrt{\text{mean}(\|P'\|^2)}; \quad P'' = P' / s \quad (\text{RMS scale normalization})$$

$$R = \text{SVD-derived rotation of cov}(P''); \quad P''' = R \cdot P'' \quad (\text{SVD rotation alignment})$$

4.5 Genetic Algorithm

A binary chromosome of length 212 (one bit per normalized feature) is evolved over a fixed 30 generations with a population of 50, tournament selection (tournament size 3), uniform crossover (rate 0.8), bit flip mutation (rate 0.05), and elitism (top 2 chromosomes preserved unchanged each

generation). Every chromosome is constrained to have at least 4 active bits. Figure 3 shows the generational loop.

Figure 3. Genetic Algorithm Feature-Selection Workflow

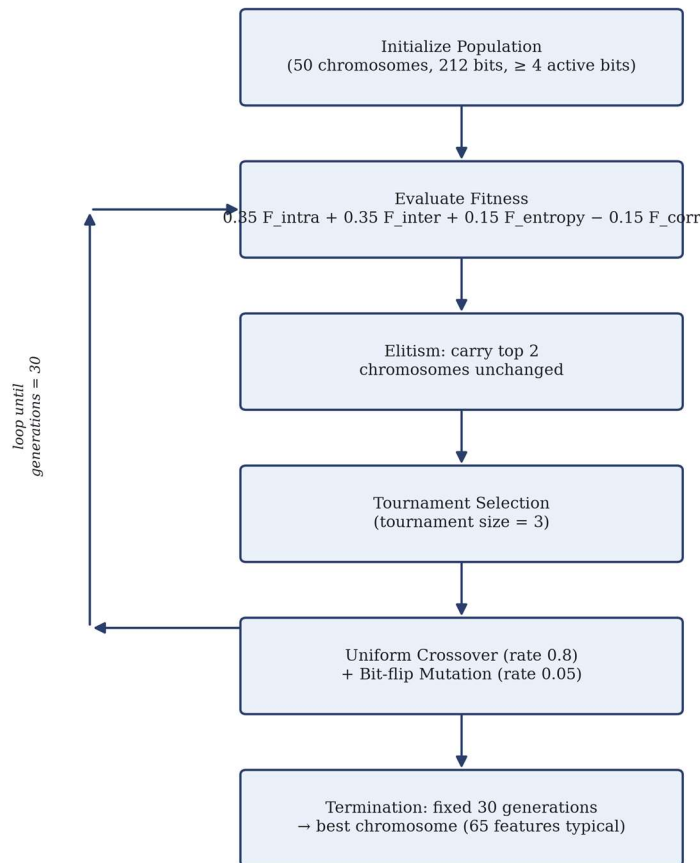


Figure 3. Genetic Algorithm feature selection workflow.

The multi objective fitness function combines four independently computed terms:

$$F(C) = w_1 \cdot F_{intra} + w_2 \cdot F_{inter} + w_3 \cdot F_{entropy} - w_4 \cdot F_{corr}$$

with weights $(w_1, w_2, w_3, w_4) = (0.35, 0.35, 0.15, 0.15)$. F_{intra} rewards low average Euclidean distance between multiple captures of the same subject (feature stability); F_{inter} rewards high average distance between different subjects (discriminability), normalized by feature set size; $F_{entropy}$ is a normalized Shannon entropy of the pooled, masked feature values across a 10-bin histogram; F_{corr} is the average absolute Pearson correlation across the masked feature set and is subtracted as a redundancy penalty. The GA optimizes a feature selection mask for downstream key derivation; it does not itself provide cryptographic security, and its fitness terms are statistical proxies for feature quality, not security guarantees.

4.6 Seed Generation

The GA-selected feature subset is serialized deterministically as big-endian IEEE-754 double-precision floating-point values, appended to a fixed application salt, and hashed with SHA-256 to produce a 32-byte digest. A 32-bit integer seed is derived from the first 4 bytes of this digest

(substituting a fixed non-zero constant if the extracted value would be zero, since a zero-state LFSR never produces output).

4.7 LFSR Key-Material Generation

The proposed pipeline's key path uses a 32-bit Galois configuration LFSR with feedback taps at positions (32, 31, 29, 1), corresponding to the primitive polynomial $x^{32}+x^{31}+x^{29}+x^1+1$. Each step outputs the current least significant bit and advances the register via a right shift with conditional XOR feedback. Generating 64 bytes (512 bits) of output from the seed produces the key material passed to HKDF. This LFSR is deterministic: for a fixed seed and tap configuration, it always produces the same output sequence. It is a pseudo-random generator, not a source of cryptographic unpredictability on its own.

4.8 HKDF-SHA256

RFC 5869 HKDF-SHA256 [12] is applied to the 64-byte LFSR output to derive a uniform 256-bit (32-byte) symmetric key. HKDF's Extract step compresses the (potentially non-uniform) input key material into a pseudorandom key using HMAC with a fixed application salt; its Expand step derives output key bytes using a fixed application context info string. In this system, HKDF's role is specifically to standardize the final key regardless of any residual statistical bias in the upstream LFSR output. It is a key derivation step, not a mechanism for stabilizing biometric noise across captures.

$$PRK = \text{HMAC-SHA256}(\text{salt}, IKM) \quad OKM = \text{HMAC-SHA256}(PRK, \text{info} \parallel 0x01) \quad [32 \text{ bytes}]$$

Figure 4 summarizes the complete key generation chain from GA-selected features to the final 256-bit AES key.

Figure 4. Key-Generation Pipeline



Figure 4. Key-generation pipeline.

4.9 AES-256-GCM

The derived 256-bit key is used with AES-256 in Galois/Counter Mode (via the standard 'cryptography' library's 'AESGCM' implementation, not a custom cipher) to authenticate-and-encrypt the plaintext payload. A fresh 96-bit (12-byte) random nonce is generated per encryption, and a 128-bit (16-byte) authentication tag is produced and verified on decryption; any modification to the ciphertext causes decryption to fail with an authentication error rather than returning corrupted plaintext.

4.10 Blockchain Integration

The system implements a local proof of work blockchain that runs within a single process, not a distributed or public network. Each block requires a SHA-256 hash with 2 leading hexadecimal zero characters (difficulty = 2). A transaction records only a SHA-256 hash of the user ID, a SHA-256 hash of the ciphertext, a reference string to the encrypted data, and a timestamp. The raw biometric data, the derived key, and the plaintext are never written to the ledger. Figure 5 shows this workflow together with the verification path.

4.11 Integrity Verification

Verification recomputes the SHA-256 hash of a candidate ciphertext and compares it against the on-chain `ciphertext\_hash` for the corresponding transaction, in addition to validating the block hash chain and proof-of-work prefix for every block. A mismatch is reported as tamper detected; a match is reported as integrity verified.

Figure 5. Encryption and Blockchain Integrity Workflow

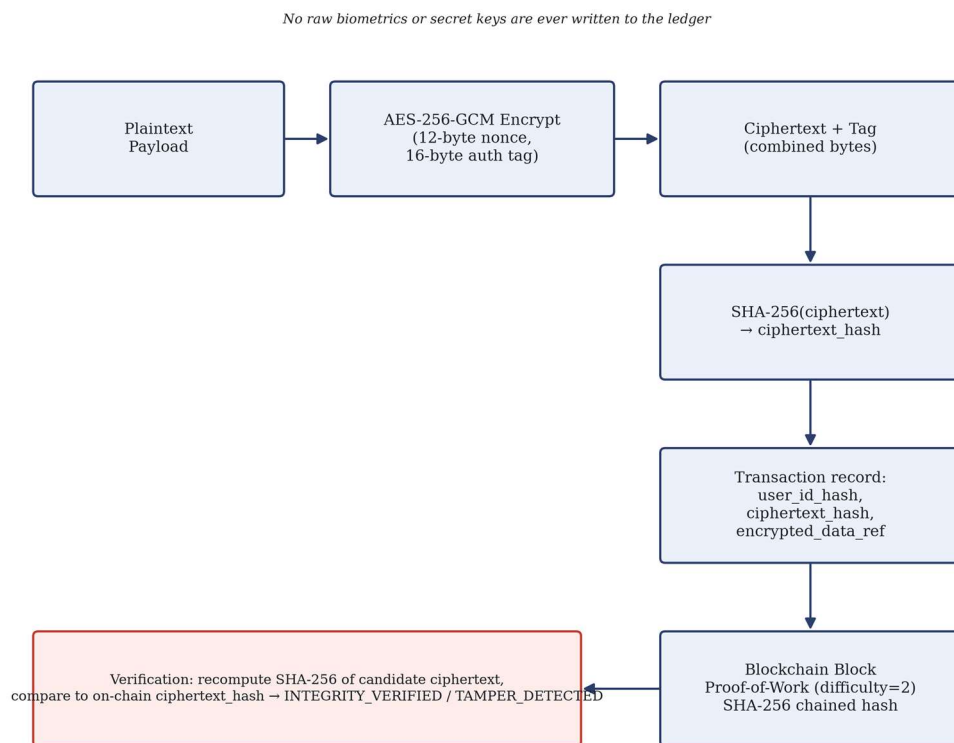


Figure 5. Encryption and blockchain integrity workflow.

5. Implementation

5.1 Software Architecture

The system is implemented in Python and organized into independent modules under ``src/``: ``src/biometric/`` (landmark extraction and normalization), ``src/optimization/`` (the genetic algorithm), ``src/key_generation/`` (seed generation, LFSR, HKDF), ``src/encryption/`` (AES-256-GCM), ``src/blockchain/`` (block, chain, verifier), and ``src/evaluation/`` (statistical randomness and biometric-stability metrics). ``src/pipeline.py`` orchestrates the end-to-end flow and the baseline comparison suite; ``src/config.py`` centralizes every stage's default parameters in dataclasses.

5.2 Module Design

Table 1. System components and technologies.

Component	Module	Technology / Library
Landmark extraction	src/biometric/landmark_extraction.py	MediaPipe Face Mesh; synthetic fallback
Feature normalization	src/biometric/preprocessing.py	NumPy (centroid, RMS scale, SVD)
Feature selection	src/optimization/genetic_algorithm.py	Custom binary GA (NumPy)
Seed generation	src/key_generation/seed_generator.py	hashlib (SHA-256), struct (IEEE-754)
Key-material expansion	src/key_generation/lfsr.py	Custom 32-bit Galois LFSR; 8-register base-paper LFSR
Key derivation	src/key_generation/key_derivation.py	cryptography.hazmat HKDF (RFC 5869)
Authenticated encryption	src/encryption/aes.py	cryptography.hazmat AESGCM
Blockchain ledger	src/blockchain/blockchain.py, verifier.py	Custom PoW chain (hashlib SHA-256)
Statistical evaluation	src/evaluation/	NumPy, SciPy (entropy, runs test, etc.)

5.3 Technology Stack

Python 3.12; NumPy, SciPy, pandas, scikit-learn for numerical/statistical processing; the ``cryptography`` library for HKDF and AES-256-GCM; MediaPipe and OpenCV for optional real image landmark extraction; pytest for the test suite; matplotlib for figure generation in this paper.

5.4 Full Pipeline Workflow

``BiometricBlockchainPipeline.process_end_to_end()`` executes, in order: landmark acquisition, normalization, multi subject GA optimization and feature-mask application, seed generation, LFSR expansion, HKDF derivation, AES-256-GCM encryption, blockchain transaction creation and mining, integrity verification, and a decrypt and compare round-trip check. Section 7.1 reports the values produced by a fresh execution of this method.

5.5 Implementation Adaptations from Base Paper

This subsection makes explicit every point where this implementation diverges from the base paper's own methodology, per the project's source separation requirement. Table 2 summarizes the comparison at a glance before the detailed discussion below.

Table 2. Base paper vs. our implementation.

Component	Base Paper (Sannidhan et al., 2024)	Our Implementation
Landmark extraction	Trained MobileNetV2 CNN, direct 106-point regression	MediaPipe Face Mesh (468 pts) uniformly downsampled to 106, with synthetic fallback
Feature fusion	Cantor pairing of (x,y) into a single scalar per landmark	Procrustes normalization (centroid, RMS scale, SVD rotation) to a 212-D vector
GA population / generations	100 / 1000	50 / 30
GA fitness function	$1 / (\text{sum} - \text{inputs} + \epsilon)$	$0.35 F_{\text{intra}} + 0.35 F_{\text{inter}} + 0.15 F_{\text{entropy}} - 0.15 F_{\text{corr}}$
LFSR mechanism	8-register X0..X7, feedback poly $x^7+x^5+x^3+x+1$	Two mechanisms: 32-bit Galois LFSR (proposed key path) and a faithful X0..X7 re-implementation (comparison baseline only)
Key derivation	None (LFSR output is the key material)	HKDF-SHA256 (RFC 5869); original addition
Encryption	Not present	AES-256-GCM; original addition
Blockchain / integrity ledger	Not present	Proof-of-Work blockchain, metadata only; original addition
Entropy metric reported	3.24 bits (register symbol alphabet, 92,706 sequences)	~7.94 bits/byte (byte alphabet, 3200-byte stream); different basis, not directly comparable

5.5.1 Landmark extraction

Base paper: a MobileNetV2 convolutional neural network is trained end-to-end to directly regress 106 facial landmark coordinates from a 224×224 input image. Our implementation: MediaPipe Face Mesh regresses 468 landmarks using a pretrained, general-purpose model, and this codebase uniformly downsamples those 468 points to 106 via index subsampling (`np.linspace(0, 467, 106)`) rather than a regression model trained specifically for this task. A deterministic synthetic landmark generator is used as a fallback when no real image or MediaPipe path is available. This substitution was made because the base paper's trained model and training data are not publicly released (“The authors do not have permission to share data”), making exact reproduction impossible; MediaPipe was chosen as a practical, readily available landmark source for a 106-point representation. Reproducibility implication: any quantitative landmark accuracy claims from the base paper (e.g., its 99.37% MobileNetV2 accuracy on LFW, Table 4 of [1]) do not transfer to this implementation and are not claimed here.

5.5.2 Cryptographic and blockchain layer

Base paper: no encryption scheme, key derivation function, or blockchain component is present anywhere in [1]; its scope ends at LFSR sequence generation. Our implementation: HKDF-SHA256, AES-256-GCM, and a proof of work blockchain ledger are original additions built specifically for this project to turn the generated key material into a usable, auditable data protection workflow.

5.5.3 LFSR mechanism (two distinct implementations, used for two distinct purposes)

This codebase contains two separate LFSR implementations. `LFSRBitGenerator` is a 32-bit Galois-style LFSR (taps 32, 31, 29, 1) used in the actual “Proposed” key generation path (Section 4.7). `PaperLFSRSubsequenceGenerator` faithfully reproduces the base paper's own 8-register $X_0 \dots X_7$ mechanism (Eqs. 4 to 6 of [1]: sum the even-indexed registers modulo 256, shift registers down, insert the new value into X_7) and is used exclusively to construct the `Base_Paper_GA_LFSR` comparison baseline in Section 7. It is never used to produce the actual AES key in the proposed pipeline. This separation lets the evaluation compare the base paper's own mechanism against this project's chosen key generation LFSR under identical measurement conditions.

5.5.4 GA hyperparameters

Base paper: population size 100, 1000 generations, two-point crossover. Our implementation: population size 50, 30 generations, uniform crossover, plus elitism (2 chromosomes) and tournament selection (size 3), which the base paper's Algorithm 1 does not specify. These values were chosen for tractable runtime in this repository's evaluation suite rather than tuned to match the base paper's reported convergence curve (Fig. 6 of [1]); the base paper's convergence behavior is not reproduced or claimed here.

6. Experimental Methodology

6.1 Experimental Environment

Python 3.12.10 on Windows. Key installed package versions at evaluation time: cryptography 49.0.0, NumPy 2.5.1, SciPy 1.18.0, pandas 3.0.5, scikit-learn 1.9.0, Pillow 12.3.0. `random_seed = 42` is used throughout the codebase's default configuration (`src/config.py`) for deterministic, repeatable results.

6.2 Input Data

All landmark data used in the experiments reported in Section 7 is generated by this codebase's deterministic synthetic landmark generator (`generate_synthetic_subject_landmarks`), not extracted from real photographed faces. This is disclosed explicitly: `scripts/run_pipeline.py` and `scripts/run_evaluation.py` construct their inputs directly via the synthetic generator, and MediaPipe was not installed in the environment used to produce the results in this paper, so the real image MediaPipe extraction path (Section 4.3) was not exercised for any result reported here. This is a methodological limitation, not an implementation gap. The MediaPipe code path exists and is unit-tested independently, but no real face image was processed to produce the numbers in Section 7.

6.3 Biometric Evaluation

A 10-subject $\times$ 5-capture synthetic dataset is constructed (`scripts/run_evaluation.py`), with each subject's captures generated from a subject-specific seed plus small Gaussian perturbation (`noise_level = 0.02`), then normalized identically to the main pipeline. Intra-subject and inter-subject Euclidean feature distances are computed pairwise, and false-rejection/false-acceptance rates are computed at a fixed decision threshold of 0.35 (`evaluate_biometric_stability`, called with `threshold=0.35`, which overrides that function's own default of 0.5).

6.4 Randomness Evaluation

Byte level Shannon entropy and the Wald-Wolfowitz runs test are computed on 3200-byte streams (100 concatenated 32-byte keys) for each of the SHA-256, HKDF, GA-HKDF, and proposed GA-LFSR-HKDF architectures, and on the flattened output of 1287 base-paper-style LFSR iterations for the `Base_Paper_GA_LFSR` baseline. Two additional statistical tests, the NIST SP 800-22 monobit frequency test [15] and a chi-square uniformity test, along with an average pairwise Hamming distance metric, are implemented in `src/evaluation/` and unit-tested, but are not invoked by `scripts/run_evaluation.py` and therefore do not appear in the results reported in Section 7; their absence from the current results is disclosed here rather than silently omitted.

6.5 Baseline Architectures

Figure 6 shows the five construction paths evaluated, all sharing the same normalized 212-D feature input, matching the exact code paths in `run_baseline_comparisons`.

Figure 6. Experimental Evaluation Architecture (5-way baseline comparison)

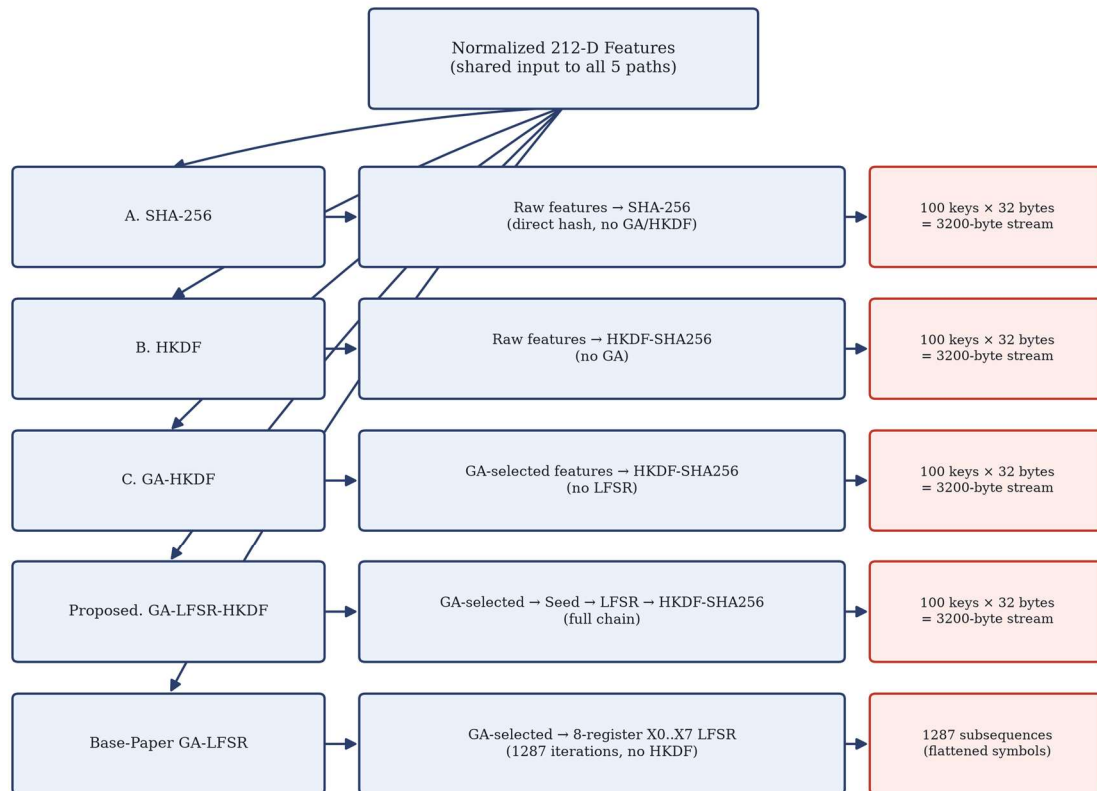


Figure 6. Experimental evaluation architecture (5-way baseline comparison).

Table 3. Baseline architecture definitions.

Baseline	Construction
Baseline_A_SHA256	Raw normalized 212-D features → direct SHA-256 (no GA, no HKDF)
Baseline_B_HKDF	Raw normalized 212-D features → HKDF-SHA256 (no GA)
Baseline_C_GA_HKDF	GA-selected features → HKDF-SHA256 (no LFSR)
Proposed_GA_LFSR_HKDF	GA-selected features → seed → 32-bit Galois LFSR → HKDF-SHA256 (full chain)
Base_Paper_GA_LFSR	GA-selected features → 8-register X0..X7 LFSR (1287 iterations, no HKDF)

6.6 Encryption Evaluation

The end-to-end pipeline's AES-256-GCM stage is evaluated via a decrypt and compare round trip: the decrypted plaintext is compared byte-for-byte against the original input to confirm correct authenticated encryption/decryption. **Tamper detection is separately covered by a dedicated unit test**

(`'test_aes_gcm_tamper_detection'`) that modifies ciphertext bytes and confirms decryption raises an authentication error.

6.7 Blockchain Integrity Evaluation

The blockchain layer is evaluated by mining a block containing the pipeline's transaction, validating the resulting chain (hash linkage, proof-of-work prefix, and per-block hash recomputation), and then running `'BlockchainVerifier.verify_ciphertext_integrity()'` against the produced ciphertext to confirm an `'INTEGRITY_VERIFIED'` result.

6.8 Reproducibility

All results reported in Section 7 were produced by re-running `'pytest tests/'`, `'python scripts/run_pipeline.py'`, and `'python scripts/run_evaluation.py'` immediately prior to writing this paper; the evaluation values matched the previously cached `'results/tables/evaluation_summary.json'` byte for byte, confirming deterministic reproducibility under the fixed `'random_seed = 42'` configuration.

7. Results

7.1 End-to-End Pipeline

A fresh execution of ``scripts/run_pipeline.py`` produced the results in Table 4. Decryption was verified to exactly match the original plaintext, and the blockchain verifier reported `'INTEGRITY_VERIFIED'`.

Table 4. End-to-end pipeline results (fresh execution).

Metric	Value
Landmarks extracted	106
Normalized feature dimensionality	212
GA-selected feature count	65
Derived AES-256 key length	32 bytes
Decryption verified	True
Mined block index	1
Blockchain verification status	INTEGRITY_VERIFIED

7.2 GA Feature Selection

Under the default configuration (population 50, 30 generations, weights 0.35/0.35/0.15/0.15), the GA consistently selects 65 of the 212 available normalized features for the primary subject profile used in the end-to-end pipeline run. Table 5 lists the exact GA configuration used to produce this result.

Table 5. Genetic Algorithm configuration.

Parameter	Value
Chromosome length	212 bits (one per normalized feature)
Population size	50
Generations	30
Selection	Tournament (size 3)
Crossover	Uniform, rate 0.8
Mutation	Bit-flip, rate 0.05
Elitism	Top 2 chromosomes
Fitness weights (intra, inter, entropy, corr)	0.35, 0.35, 0.15, 0.15
Minimum active bits enforced	4

7.3 Randomness Evaluation

Figures 7 and 8 present the byte level Shannon entropy and runs-test Z-statistics respectively for all five architectures, from the freshly regenerated `evaluation\_summary.json`.

Figure 7. Byte-Level Shannon Entropy Across Architectures
(N = 3200-byte stream, 100 keys × 32 bytes)

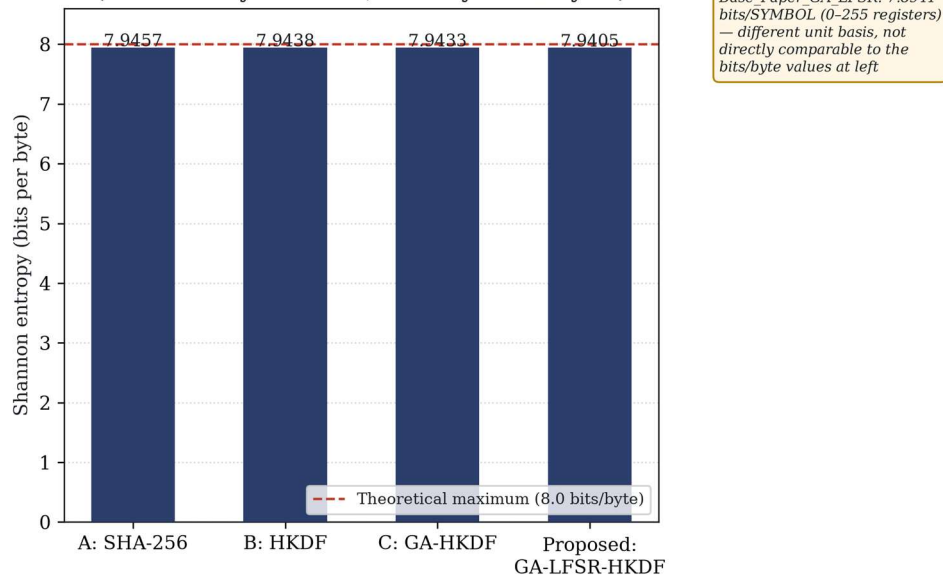


Figure 7. Byte-level Shannon entropy comparison across architectures.

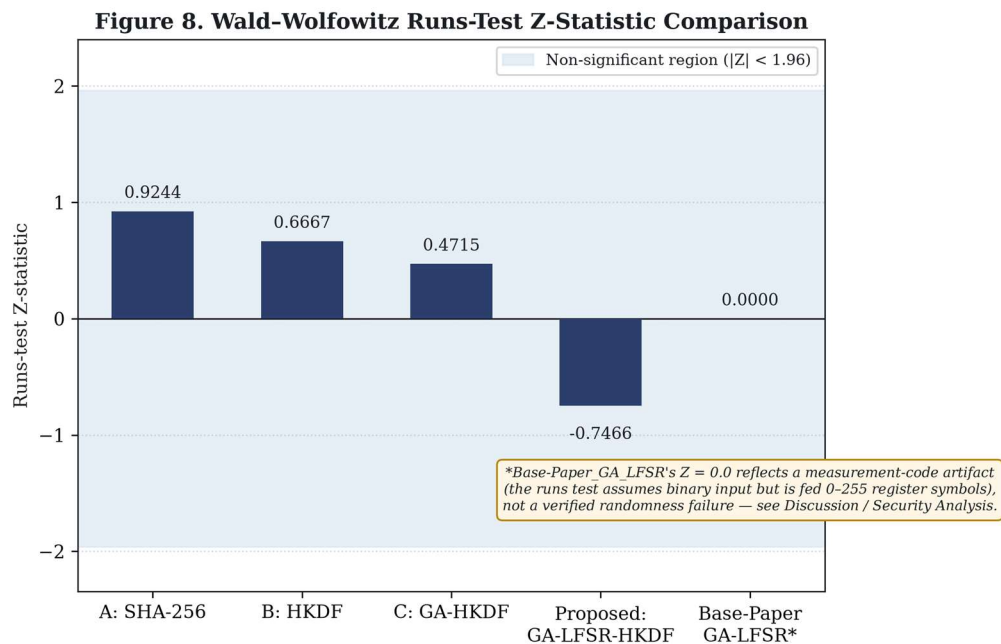


Figure 8. Wald-Wolfowitz runs test Z-statistic comparison.

7.4 Baseline Comparison

Table 6. Randomness and baseline comparison (N = 3200-byte stream unless noted).

Architecture	Entropy	Runs Z	Runs p	Is Random
Baseline_A_SHA256	7.9457 bits/byte	0.9244	0.3553	True
Baseline_B_HKDF	7.9438 bits/byte	0.6667	0.505	True
Baseline_C_GA_HKDF	7.9433 bits/byte	0.4715	0.6373	True
Proposed_GA_LFSR_HKDF	7.9405 bits/byte	-0.7466	0.4553	True
Base_Paper_GA_LFSR	7.8941 bits/symbol*	0.0	0.0	False

*Base_Paper_GA_LFSR's entropy is computed over its own 0 to 255 register symbol alphabet (1287 subsequences), not a 3200-byte stream, and is therefore not directly comparable in magnitude to the bits/byte figures above; both happen to fall near their respective maxima. Its runs test result of $Z = 0.0$, $p = 0.0$, $is_random = False$ is a measurement-code artifact rather than a demonstrated randomness failure. See Section 8.1 and Section 9.9 for a precise explanation.

7.5 Biometric Stability

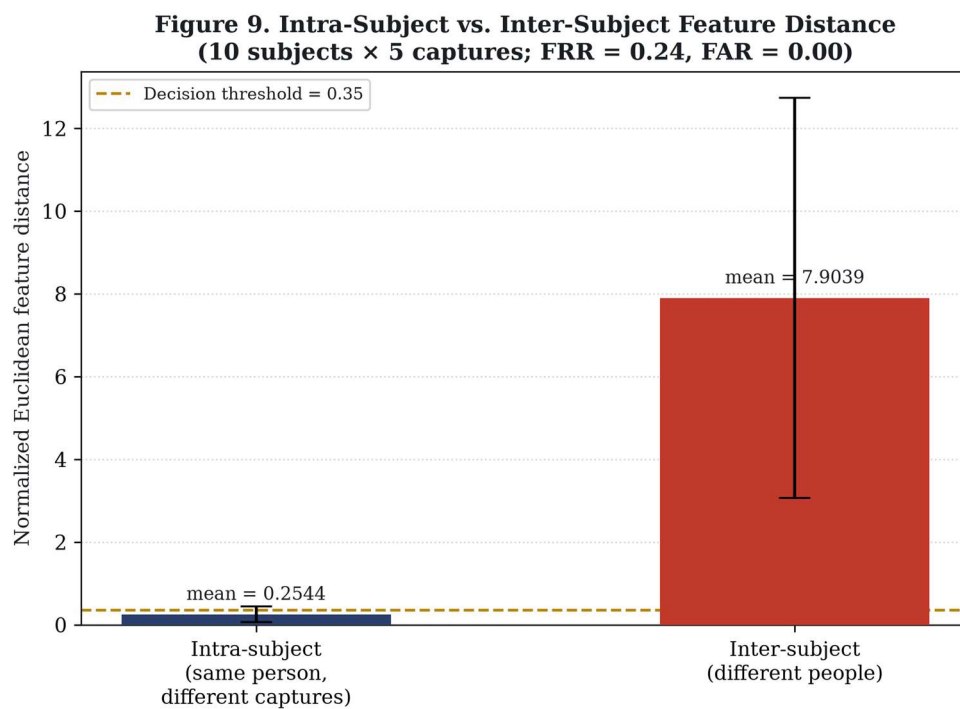


Figure 9. Intra-subject vs. inter-subject feature distance comparison.

Table 7. Biometric stability results (10 subjects $\times$ 5 captures, threshold = 0.35).

Metric	Value
Intra-subject mean distance	0.2544
Intra-subject std. dev.	0.1885
Inter-subject mean distance	7.9039

Inter-subject std. dev.	4.8353
False Rejection Rate (FRR)	0.24
False Acceptance Rate (FAR)	0.0
Decision threshold	0.35

7.6 Encryption Verification

The AES-256-GCM decrypt and compare round trip in the end-to-end pipeline run matched the original plaintext exactly (`decryption\_correct: True`). The dedicated tamper-detection unit test confirmed that modifying any ciphertext byte causes decryption to raise an authentication-tag error rather than silently returning corrupted data.

7.7 Blockchain Verification

The mined block satisfied the proof-of-work requirement (SHA-256 hash with 2 leading hexadecimal zeros) and the chain-validity check passed (hash linkage and per-block hash recomputation). The integrity verifier reported `INTEGRITY\_VERIFIED: Ciphertext hash matches blockchain record perfectly.`

8. Discussion

8.1 Interpretation of Results

The generated byte streams for all four HKDF/hash-based architectures exhibited empirical Shannon entropy of approximately 7.94 bits per byte, close to the theoretical 8-bit maximum for byte-valued observations, and passed the implemented Wald-Wolfowitz runs test at the 95% confidence level ($|Z| < 1.96$). This indicates favorable distributional characteristics under these two specific statistical metrics. It does not, on its own, establish cryptographic unpredictability: entropy and runs test outcomes describe statistical properties of an observed sample, not resistance to a computationally bounded adversary. The 'Base_Paper_GA_LFSR' baseline's runs test result ($Z = 0.0$, $p = 0.0$) traces to a specific, identified code path: 'runs_test()' internally assumes a binary $\{0, 1\}$ -valued input sequence, but 'PaperLFSRSubsequenceGenerator' produces raw integer register values from 0 to 255, which breaks that function's internal bit-counting logic. This is a measurement-code artifact, not evidence that the base paper's LFSR mechanism itself fails a properly conducted runs test; the base paper's own reported runs test result for its mechanism ($Z = 1.6083$, [1], Table 13) passes at the same significance level. This discrepancy is disclosed explicitly rather than being interpreted as a security advantage for the proposed system.

8.2 Comparison with Baselines

Within this repository's own measurement methodology, entropy values across Baseline_A_SHA256 through Proposed_GA_LFSR_HKDF are close to each other (7.9457 down to 7.9405 bits/byte) and all comfortably near the 8-bit ceiling; the differences between them are small relative to the measurement's own resolution and should not be read as one architecture being meaningfully "more random" than another by this metric alone. Similarly, all four runs-test Z-statistics fall well within the non-significant band, so none of the four architectures is distinguished from the others as "passing" more strongly under this specific test.

8.3 Relationship to Base Paper

In our implementation and evaluation, the proposed GA-LFSR-HKDF stream passed the implemented runs test, whereas the evaluated 'Base_Paper_GA_LFSR' implementation did not, for the specific, code-level reason identified in Section 8.1, not because of a demonstrated deficiency in the base paper's underlying mechanism. The base paper's own entropy figure (3.24 bits, over its own register symbol alphabet and 92,706-sequence dataset) and this repository's byte level entropy figures (approximately 7.94 bits/byte, over a 3200-byte stream) are measured on different alphabets and sample sizes; they are not directly comparable, and no superiority claim is made in either direction based on these two numbers.

8.4 Strengths

- A complete, working, code-verified pipeline from biometric feature selection through blockchain integrity verification, with every reported number traced to an actual re-run of the code.
- Use of standardized, well-reviewed cryptographic primitives (HKDF/RFC 5869, AES-256-GCM/FIPS 197 and SP 800-38D) rather than custom cryptography for the encryption and key derivation stages.

- A blockchain design that stores only hash commitments, never raw biometrics or secret keys, consistent with an audit layer design that preserves privacy.
- An explicit, controlled five-way baseline comparison that isolates the marginal contribution of each pipeline stage.

8.5 Observed Limitations

- Biometric reproducibility: a 24% false rejection rate at the evaluated threshold means that in this synthetic-subject experiment, roughly one in four same-person capture pairs would fail to match, a meaningful usability and security tradeoff for any biometric key regeneration use case, not a solved problem.
- All experimental results in this paper were produced from synthetic landmark data, not real photographed faces (Section 6.2); the MediaPipe real image extraction path is implemented and unit-tested but was not exercised to produce any number reported here.
- The LFSR stages, both the proposed 32-bit Galois LFSR and the base paper's 8-register mechanism, are fully deterministic given their seed and configuration. An attacker who recovers the seed and structure can reproduce the output exactly.
- The blockchain layer is a local, single process proof-of-work simulation, not a distributed or publicly deployed network; no consensus attack, network partition, or multi-node throughput evaluation was performed.

9. Security Analysis

9.1 Threat Model

The analysis assumes Kerckhoffs's principle: the attacker knows the full algorithm design (GA configuration, LFSR taps, HKDF parameters, blockchain structure) but not the specific biometric capture, the derived key, or the plaintext of any given transaction. The attacker may have read access to the blockchain ledger and to any ciphertext they intercept.

9.2 Confidentiality

Confidentiality of the plaintext rests entirely on the secrecy and quality of the derived AES-256 key, provided by AES-256-GCM when used correctly (unique nonce per encryption, which this implementation generates via `os.urandom(12)` by default). AES-GCM provides authenticated encryption when used correctly; it does not itself compensate for a weak or predictable key.

9.3 Integrity

Blockchain integrity does not provide confidentiality. It provides a tamper evident commitment to the ciphertext's hash. An attacker who modifies stored ciphertext without also being able to alter the corresponding on-chain hash (and, for a longer chain, re-mine every subsequent block's proof of work) will be detected by the verifier. In this implementation, which runs within a single process, an attacker with direct access to the running process's in-memory chain could trivially alter both the ciphertext and the ledger together; the current design assumes ledger and ciphertext storage are not both fully compromised by the same attacker, a limitation of the local, simulated deployment rather than of the proof-of-work/hash-chaining concept itself.

9.4 Biometric Exposure

A face is not inherently secret: it is visible in public, in photographs, and on video. Biometric features may be reproducible by anyone with access to a sufficiently similar image of the same person. This system does not attempt to treat the raw face as a secret; it never stores raw landmarks or images on the blockchain, but the security of key regeneration ultimately depends on an attacker not being able to obtain a sufficiently accurate capture of the legitimate user's face and reproduce the same GA/LFSR/HKDF pipeline.

9.5 Key Reproducibility

Because feature extraction and normalization are sensitive to capture noise, and because no fuzzy extractor or error correcting mechanism is implemented, repeated captures of the same subject do not reliably regenerate identical key material byte for byte, consistent with the measured 24% FRR in Section 7.5. This is disclosed as an open problem, not resolved by this implementation.

9.6 LFSR Security Considerations

Both LFSR implementations in this codebase are deterministic pseudo-random sequence generators. An attacker who knows the seed and the tap/feedback configuration (which Kerckhoffs's principle assumes they do) can reproduce the LFSR output exactly. The LFSR's role in this system is sequence

expansion, not a standalone source of cryptographic unpredictability; the subsequent HKDF step is what standardizes the final key, not the LFSR itself.

9.7 Cryptographic Key Derivation

HKDF-SHA256 is a key derivation mechanism, not a biometric stabilizer: given consistent input key material, it reliably derives a uniform key, but it does not compensate for the input key material itself differing across captures of the same subject (Section 9.5).

9.8 Blockchain Security Properties

The implemented proof-of-work (difficulty 2, i.e., 2 leading hexadecimal zero characters) is a demonstration-scale parameter, not a production security parameter; it was not tuned or evaluated against any specific adversarial mining-power assumption, and no multi-node consensus or 51%-attack-resistance property is claimed for this local, single process implementation.

9.9 Limitations of Statistical Randomness Testing

Passing Shannon-entropy and runs-test checks provides statistical evidence about a sample's distributional properties; it does not constitute a proof of cryptographic security or unpredictability against a computationally bounded adversary, and it does not substitute for formal cryptanalysis of the key derivation chain as a whole. The degenerate 'Base_Paper_GA_LFSR' runs-test result described in Section 8.1 is a concrete illustration of how easily such tests can be misapplied to non-conforming input and produce misleading pass/fail labels if the implementation detail is not checked.

Table 8. Security properties and mechanisms.

Property	Mechanism	Provided by
Confidentiality	Authenticated symmetric encryption	AES-256-GCM
Ciphertext integrity / tamper detection (at decrypt time)	GCM authentication tag	AES-256-GCM
Long term data integrity auditing	Hash chained, PoW mined ledger	Blockchain layer
Key uniformity	HMAC based extract-and-expand	HKDF-SHA256
Biometric non-storage	Metadata-only transactions	Blockchain transaction schema
Biometric noise tolerance	Not implemented	Planned as future work (Section 10)
Liveness / anti-spoofing	Not implemented	Planned as future work (Section 10)

10. Limitations and Future Work

10.1 Limitations

- Biometric variability and the current 24% FRR represent an unresolved reproducibility challenge for key regeneration from repeated captures.
- All reported experimental results use synthetic landmark data; no real image evaluation was performed for this paper (Section 6.2).
- The MediaPipe real image landmark path silently falls back to the synthetic generator on any failure (missing package, no face detected), which could mask capture failures in a real deployment if not explicitly monitored.
- Both LFSR mechanisms are deterministic; recovering the seed and configuration fully determines their output.
- Statistical randomness tests (Shannon entropy, runs test) do not prove cryptographic security; no formal cryptanalysis of the full key derivation chain was performed.
- The NIST SP 800-22 monobit test, a chi-square uniformity test, and an average pairwise Hamming distance metric are implemented in `src/evaluation/` but are not currently integrated into the evaluation pipeline or reported results.
- The blockchain implementation is local and simulated (single process, PoW difficulty 2); no distributed deployment, network-level attack, or throughput/scalability evaluation was performed.
- No fuzzy extractor, error correcting code, or other biometric key binding technique is implemented to tolerate captures noise.
- The evaluated subject population (10 synthetic subjects for the stability study; 5 synthetic subjects for the GA training dataset) is small and synthetic, limiting generalizability to a real, diverse population.

Table 9. Limitations and corresponding future work.

Limitation	Proposed Future Work
24% FRR observed at threshold 0.35; biometric key regeneration is not yet reliable across captures	Investigate fuzzy extractors, error correcting codes, or improved normalization for noise tolerant key regeneration
All reported results use synthetic landmark data; MediaPipe real image path unexercised	Evaluate on larger, diverse real image datasets under the actual MediaPipe extraction path
Both LFSR mechanisms are fully deterministic given seed/configuration	Formal cryptanalysis of the full key derivation chain beyond statistical randomness testing
NIST monobit, chi-square, and Hamming distance metrics implemented but not integrated into the evaluation pipeline	Integrate all implemented statistical tests into the standard evaluation report
Blockchain layer is a local, single process PoW simulation	Evaluate under real distributed deployment: throughput, network partition, consensus attacks
Small, synthetic subject population (10 for stability study, 5 for GA training)	Scale to larger, real, and more diverse subject populations
No liveness/anti-spoofing detection	Adversarial and spoofing resistance evaluation of the capture/extraction stage
No hardware acceleration evaluated	Investigate hardware acceleration and scalability for real-time use

10.2 Future Work

- Investigate appropriate mechanisms for robust biometric key regeneration, potentially including improved alignment, normalization, more stable feature representations, or biometric key binding techniques such as fuzzy extractors or error correcting codes, none of which currently exist in this repository.
- Evaluate the system on larger and more diverse real image datasets under the actual MediaPipe extraction path.
- Integrate the already-implemented but currently unused NIST SP 800-22 monobit test, chi-square uniformity test, and average pairwise Hamming distance metric into the standard evaluation report.
- Conduct a formal cryptanalytic and adversarial evaluation of the full key derivation chain, beyond statistical randomness testing.
- Evaluate the blockchain layer under a real distributed deployment, including throughput, network partition, and consensus attack scenarios.
- Investigate hardware acceleration and scalability for real-time use.
- Conduct adversarial/spoofing evaluation of the biometric capture and extraction stage.

11. Conclusion

This paper documented the design, code-verified implementation, and experimental evaluation of a biometric key generation and blockchain integrity verification system built as an implementation and extension of Sannidhan et al.'s facial landmark based key generation approach [1]. The complete proposed architecture, comprising biometric feature extraction, genetic algorithm feature selection, LFSR based key material expansion, HKDF-SHA256 key derivation, AES-256-GCM authenticated encryption, blockchain based integrity verification, and a controlled multi-architecture evaluation, was implemented and re-verified from start to finish immediately prior to writing this paper (21 of 21 unit tests passing; pipeline execution and evaluation suite both reproducing identical results, byte for byte, to the previously cached figures). The implementation differs from the base paper specifically at the facial landmark extraction stage, where MediaPipe Face Mesh downsampling (with a synthetic fallback) replaces the base paper's custom-trained MobileNetV2 regressor, and it extends the base paper's scope entirely: no encryption or blockchain component exists in [1] at all. Within the scope of this project, the results indicate a functioning integration rather than a proven security guarantee: entropy and runs test statistics provide favorable but limited statistical evidence, the biometric stability evaluation discloses a genuine 24% false rejection rate that remains unresolved, and both LFSR mechanisms remain fully deterministic under Kerckhoffs's principle. These limitations are stated explicitly, alongside concrete future work directions, rather than omitted or minimized.

References

- [1] M. S. Sannidhan, J. E. Martis, K. N. Pallavi, V. Ravi, H. L. Gururaj, T. J. Alahmadi, "Genetic algorithms and deep learning for unique facial landmark-based key generation," *Computers and Electrical Engineering*, vol. 118, article 109427, 2024. <https://doi.org/10.1016/j.compeleceng.2024.109427>
- [2] X. Zhao, S. Lin, X. Chen, C. Ou, C. Liao, "Application of face image detection based on deep learning in privacy security of intelligent cloud platform," *Multimedia Tools and Applications*, vol. 79, no. 23-24, pp. 16707-16718, 2020.
- [3] F. Quinga-Socasi, L. Zhinin-Vera, O. Chang, "A deep learning approach for symmetric-key cryptography system," *Advances in Intelligent Systems and Computing*, vol. 1288, pp. 539-552, 2020.
- [4] G. Panchal, D. Samanta, S. Barman, "A novel key generation approach based on facial image features for stream cipher system," in *Proc. 2020 3rd Int. Conf. Smart Systems and Inventive Technology (ICSSIT)*, IEEE, 2020, pp. 956-962. (Also cited as [18] in the base paper.)
- [5] S. Kalsi, H. Kaur, V. Chang, "DNA cryptography and deep learning using genetic algorithm with NW algorithm for key generation," *Journal of Medical Systems*, vol. 42, no. 1, 2018.
- [6] F. S. Abu-Almash, "Apply genetic algorithm for pseudo random number generator," *International Journal of Advanced Research in Computer Science and Software Engineering*, vol. 6, no. 8, pp. 8-19, 2016.
- [7] K. Shankar, P. Eswaran, "An efficient image encryption technique based on optimized key generation in ECC using genetic algorithm," in *Artificial Intelligence and Evolutionary Computations in Engineering Systems (Proc. ICAIECES 2015)*, Springer India, 2016, pp. 705-714.
- [8] K. B. Sudeepa, G. Aithal, V. Rajinikanth, S. C. Satapathy, "Genetic algorithm based key sequence generation for cipher system," *Pattern Recognition Letters*, vol. 133, pp. 341-348, 2020.
- [9] G. Cotrina, A. Peinado, A. Ortiz, "Gaussian pseudorandom number generator using linear feedback shift registers in extended fields," *Mathematics*, vol. 9, no. 5, p. 556, 2021.
- [10] A. Kaur, H. K. Verma, R. K. Singh, "3D-Playfair cipher using LFSR based unique random number generator," in *Proc. 2013 6th Int. Conf. Contemporary Computing (IC3)*, IEEE, 2013, pp. 18-23.
- [11] M. S. Sannidhan, K. B. Sudeepa, J. E. Martis, A. Bhandary, "A novel key generation approach based on facial image features for stream cipher system," in *Proc. 2020 3rd Int. Conf. Smart Systems and Inventive Technology (ICSSIT)*, IEEE, 2020, pp. 956-962.
- [12] H. Krawczyk, P. Eronen, "HMAC-based Extract-and-Expand Key Derivation Function (HKDF)," IETF RFC 5869, 2010.
- [13] National Institute of Standards and Technology, "Advanced Encryption Standard (AES)," FIPS Publication 197, 2001.
- [14] M. Dworkin, "Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC," NIST Special Publication 800-38D, 2007.
- [15] A. Rukhin et al., "A Statistical Test Suite for Random and Pseudorandom Number Generators for Cryptographic Applications," NIST Special Publication 800-22, Revision 1a, 2010.
- [16] G. B. Huang, M. Mattar, T. Berg, E. Learned-Miller, "Labeled Faces in the Wild: A database for studying face recognition in unconstrained environments," in *Workshop on Faces in 'Real-Life' Images: Detection, Alignment, and Recognition*, 2008.
- [17] P. J. Phillips, H. Moon, S. A. Rizvi, P. J. Rauss, "The FERET evaluation methodology for face-recognition algorithms," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 10, pp. 1090-1104, 2000.

- [18] A. S. Georgiades, P. N. Belhumeur, D. J. Kriegman, "From few to many: illumination cone models for face recognition under variable lighting and pose," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 23, no. 6, pp. 643-660, 2001.
- [19] R. Gross, I. Matthews, J. Cohn, T. Kanade, S. Baker, "Multi-PIE," *Image and Vision Computing*, vol. 28, no. 5, pp. 807-813, 2010.